Daffodil International University
Department of Software Engineering

System Design Document

Unit 8: Frontend

TrustBazaar: A Verified Peer-to-Peer Auction and Marketplace Platform for Used Goods

S. M. Nihal Ahmed (232-35-002)
Afrim Hossen Khan (232-35-076)
Sabikun Nahar Sinthia (232-35-475)

Status	Draft for team review
Version	0.1, 18 October 2026
Implements	SRS v2.0 (`docs/srs.tex`)
Depends on	Units 1 to 7 (their HTTP APIs, live events and rules)
Reference code	`docs/design/web/`, tested in Node, Chromium and nginx (83 checks, Appendix B)

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Requirements covered . . . 3
1.4 The repository today . . . 3

2 Overview **4**
2.1 Responsibilities . . . 4
2.2 Architecture . . . 4
2.3 Technology . . . 4

3 Application Structure **5**
3.1 Four documents . . . 5
3.2 Code layout . . . 5
3.3 Routes of the user app . . . 5

4 Talking to the Backend **6**
4.1 One origin . . . 6
4.2 The API client . . . 6
4.3 Server state . . . 8
4.4 The live connection . . . 8
4.5 The server clock and countdowns . . . 8

5 Screens **9**
5.1 Account and sign-in (IF-UI-01) . . . 9
5.2 KYC wizard (IF-UI-02, NFR-USE-02) . . . 9
5.3 Listings, search and the auction panel (IF-UI-03, NFR-USE-01) . . . 10
5.4 Checkout and payment (IF-UI-04) . . . 10
5.5 The order view (IF-UI-05, NFR-USE-07) . . . 11
5.6 Game accounts (IF-UI-06, NFR-USE-03) . . . 11
5.7 Chat (IF-UI-07) . . . 11
5.8 Disputes and ratings (Unit 7) . . . 12
5.9 Seller dashboard (IF-UI-08) . . . 12
5.10 Staff console (IF-UI-09) . . . 12
5.11 Consequences before confirmation (IF-UI-10) . . . 12

6 Language, Money and Time **13**
6.1 Bangla and English (NFR-USE-04) . . . 13

6.2 Money . . . 13
6.3 Dates, times and deadlines (NFR-USE-08) . . . 13
6.4 The money state in plain language (NFR-USE-07) . . . 14

7 Accessibility and Small Screens **14**

8 Security **15**
8.1 Security headers per document . . . 15
8.2 Other client-side rules . . . 15

9 Performance **16**

10 Build and the Web Container **16**

11 Parameters **16**

12 Testing **17**

13 Notes for Integration **18**

A Traceability **19**

B Reference Code and Tests **20**
B.1 Test results . . . 20
B.2 `src/api.js` . . . 21
B.3 `src/money.js` . . . 23
B.4 `src/time.js` . . . 24
B.5 `src/moneyState.js` . . . 26
B.6 `src/live.js` . . . 27
B.7 `src/i18n.js` and `src/redirect.js` . . . 28
B.8 `src/messages/en.json` . . . 29
B.9 `container/nginx.conf` and the credential pages' headers . . . 30

1 Introduction

1.1 Purpose

This document is the eighth unit of TrustBazaar's System Design Document: the web client. Units 1 to 7 designed the backend, each with its HTTP API, its live events and the rules the client must follow. This unit decides how one React application turns them into the screens of the SRS (IF-UI-01 to IF-UI-10). It covers:

- the structure of the application: four HTML documents (the user app, the staff console, the KYC capture page and the credential pages), routing, and the folders per unit;
- how the client talks to the backend: one API client that follows Unit 1's cookie, CSRF, refresh and step-up rules; server-state caching; the live connection and its resynchronisation; and the server clock behind every countdown;
- every screen of IF-UI-01 to IF-UI-10, with the API calls it makes and the rules it must show;
- Bangla and English, money and Dhaka time; accessibility and 360-pixel screens; security headers per document; performance; and the build and the web container.

Like Unit 5's `sealed.js`, this unit ships tested reference code (`docs/design/web/`, Appendix B). It contains the parts where a mistake would cost money or trust, and which can be tested without the screens: the API client, money display and input, Dhaka time and countdowns, the server clock, the plain-language money state, the ordering of live updates, and the two message catalogues. The application imports these modules unchanged.

1.2 Scope

In scope: the web client in `frontend/`, its documents, routes, screens, client-side state, language, accessibility, security headers for its documents, its build, and the web container of NFR-MNT-01.

Out of scope: every rule the server enforces. The client shows rules and checks input early so the user gets an answer at once, but it never decides anything: all authorisation, amounts, deadlines and state changes are the backend's (SEC-ACC-01, SEC-PAY-06). The full system architecture and the Compose file are Unit 9.

1.3 Requirements covered

- **Interface:** IF-UI-01 to IF-UI-10; IF-COM-01 and IF-COM-02 (client side); IF-SW-07 and IF-SW-08 (Web Cryptography and Geolocation APIs).
- **Usability:** NFR-USE-01 to NFR-USE-08.
- **Performance:** NFR-PERF-07 (LCP), and the client's share of NFR-PERF-03 and NFR-PERF-04.
- **Security:** SEC-SES-03, SEC-SES-04 and SEC-SES-07 (client side); SEC-INP-03; SEC-WEB-01 to SEC-WEB-04; SEC-GAM-03; SEC-ACC-05 (masking in the console).
- **Other:** NFR-MNT-01 (web container), NFR-MNT-07 (ESLint), NFR-CMP-05 (simulated payments and couriers shown on screen); the "Unit 8" integration notes of Units 4 to 7.

1.4 The repository today

`frontend/` is a starting skeleton: four placeholder pages, an API client that reads a bearer token from `localStorage`, and a Dockerfile that runs the Vite development server. The design replaces the client with the reference module `api.js` (Unit 1 already listed this change, because a token in `localStorage` breaks SEC-SES-03), adds the documents and routes below, and changes the

container to serve a production build (Section 10). React 18, Vite and React Router 6, which the skeleton already uses, stay.

2 Overview

2.1 Responsibilities

1. Show every amount, deadline and state exactly as the server gives it, in plain language, in Bangla or English, in Dhaka time.
2. Before any irreversible action, say what it will do (IF-UI-10), and send it once.
3. Keep live views (auctions, chat, notifications) correct after lost messages, reconnections and sleeping phones.
4. Hold no secret the browser does not need: no session token in reach of scripts, no credential outside the sealed handover.
5. Work on a 360-pixel phone on a 4G connection, with a keyboard, and with a screen reader.

2.2 Architecture

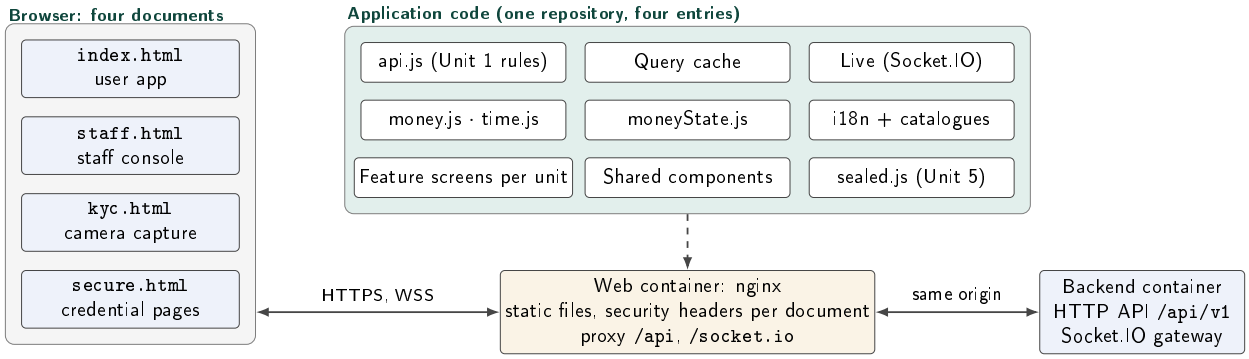


Figure 1: The web client. One code base builds four documents; the web container serves them with their own headers and forwards the API and the live connection, so browser and API share one origin.

2.3 Technology

Library	Version	Use and reason
React, React DOM	18.3.1	C-1. Escapes all text by default, which SEC-INP-03 relies on.
React Router	6.26.2	Routes with lazy-loaded screens; one router per document.
TanStack Query	5.59.0	Server state: caching, refetch on focus and reconnect, invalidation by live events. Replaces any global store: what the server knows stays the server's.
Socket.IO client	4.8.1	The live connection (C-1, IF-COM-01).
`@hpke/core`	1.9.0	Only in `secure.html`, through Unit 5's `sealed.js`; bundled, never from a CDN (SEC-GAM-03).
Vite, TypeScript	7.x, 5.5	Build (already in the repository); four HTML entries.
ESLint	9.x	NFR-MNT-07, with the security rules of Section 8.

Exact versions are pinned in `package-lock.json`. Nothing else is needed. There is no CSS framework, no date library (the browser's `Intl` does the formatting, Section 6) and no i18n library (the catalogue module is 40 lines). The page budget of NFR-PERF-07 is the reason (Section 9). Styles are CSS modules with design tokens as CSS variables.

3 Application Structure

3.1 Four documents

A single-page application is one document, and some browser protections apply per document: the Content Security Policy, the Permissions-Policy (which decides whether a page may use the camera), and whether the browser may cache the page. The SRS asks for different settings on different pages (SEC-WEB-02, SEC-GAM-03), so the client is built as four documents, each a Vite entry with its own bundle:

Document	Paths	Why it is separate
`index.html`	everything else	The user app. Geolocation is allowed, camera is not.
`staff.html`	`/staff/*`, `/admin/*`	Staff code is never sent to users. Staff sessions have their own cookies (I-51) and shorter limits (CP-03), so a staff member signed in to the console is not signed in to the user app in the same browser, and the reverse (FR-OPS-04).
`kyc.html`	`/account/kyc/*`	The only document allowed to use the camera (SEC-WEB-02, FR-KYC-02).
`secure.html`	`/secure/orders/{id}/credentials`, `/deliver`	The two credential screens of Unit 5 (SEC-GAM-03): a minimal bundle with `sealed.js` and nothing else, the strictest policy and `Cache-Control: no-store`.

Moving between documents is a normal page load. Links into a credential screen therefore always load a fresh `secure.html`, and its policy cannot be loosened by anything that ran on an earlier page.

3.2 Code layout

```
frontend/
  index.html  staff.html  kyc.html  secure.html     the four entries
  src/
    lib/        api.js money.js time.js moneyState.js live.js i18n.js   (reference modules, unchanged)
    messages/   en/*.json  bn/*.json                one catalogue file per feature, same keys in both
    shared/     components (Money, DateTime, Deadline, Countdown, ConfirmDialog, StepUpDialog, ...)
    features/   account/ kyc/ listings/ search/ auctions/ checkout/ orders/ delivery/ game/ chat/
                notifications/ disputes/ ratings/ seller/ privacy/      (one folder per unit area)
    staff/      queues/ cases/ admin/                (only in the staff.html bundle)
    entries/    app.tsx staff.tsx kyc.tsx secure.tsx
```

A feature folder holds its routes, screens, query hooks and catalogue keys. It may import `lib` and `shared`, but not another feature's internals; a cross-feature link goes through routes. The ESLint import rules check this, and that `staff/` is imported only by the staff entry.

3.3 Routes of the user app

Path	Who	Screen (IF-UI)
`/, /search`	everyone	Home and search with filters (IF-UI-03)
`/listings/:id`	everyone	Listing detail: Buy, or the live auction panel (IF-UI-03)

Path	Who	Screen (IF-UI)
`/users/:id`	everyone	Public profile with ratings and badges (FR-REV-04, 05)
`/register, /signin, /reset`	guests	IF-UI-01
`/sell/new, /sell/:id`	verified	Listing creation and editing; game details and ownership proof (IF-UI-03, IF-UI-06)
`/checkout/:listingId`	verified	IF-UI-04
`/payments/return/:provider`	buyer	Status after the provider's page; never confirms by itself (FR-PAY-02)
`/orders, /orders/:id`	parties	Order view with its tabs: handover or shipment, game account, dispute, chat (IF-UI-05, 06, 07)
`/messages, /messages/:thread`	signed in	Chat (IF-UI-07)
`/notifications`	signed in	Notification centre and preferences (FR-NTF-04, 05)
`/seller`	verified	Seller dashboard (IF-UI-08)
`/account, /account/security`	signed in	Profile; sessions, MFA, password, contact changes (IF-UI-01)
`/account/payout, /account/privacy`	signed in	Payout account (FR-PAY-17); consents, data export, closure (FR-PRV)
`/help/:topic`	everyone	Auction rules, fees, disputes, game-account risks, terms (NFR-CMP-04)

Who decides only what the client offers; for example, a guest who opens `/checkout` is sent to sign in. It is not access control. Every request is authorised by the server (SEC-ACC-01), and a screen that receives `403` or `404` shows the message for it.

The staff console's routes follow the eight queues of FR-OPS-01 (Unit 7, Section 5.10).

4 Talking to the Backend

4.1 One origin

The browser loads the documents and calls the API on one origin. In development, Vite's proxy forwards `/api` and `/socket.io` to the backend; in the web container, nginx does the same (Section 10). All cookies are therefore first-party and `SameSite=Strict` works, as Unit 1 assumed. The API base is the relative path `/api/v1`; there is no API address to configure, so the skeleton's `VITE_API_BASE_URL` goes.

4.2 The API client

Every request goes through `createApi` (`lib/api.js`, Appendix B). It follows Unit 1's rules:

Rule	What the client does
Tokens (SEC-SES-03)	None in reach of scripts. Requests use `credentials: 'same-origin'`; the browser sends the HttpOnly cookies. The client never sets an `Authorization` header (tested in Chromium).
CSRF (SEC-SES-04)	On `POST`, `PUT`, `PATCH` and `DELETE`, `X-CSRF-Token` is copied from the readable `__Host-tb_csrf` cookie. It is read on each attempt, so a refresh's new cookie is used at once.
Expired access token	On `401`, the client refreshes once and retries the request once, with the same headers. If the refresh is refused, or the retry is `401` again, the session has ended: the app clears its cache and shows "You have been signed out".

Rule	What the client does
One refresh for all tabs	Refreshes are serialised across tabs with the Web Locks API (lock `tb-refresh`). A tab that obtains the lock first checks whether another tab has already refreshed, and if so retries without refreshing. A `409` from the refresh endpoint ("rotated a moment ago", Unit 1 DV-4) is retried after 300 ms.
Step-up (SEC-AUTH-08)	A request declared with a step-up action (for example `payout.change`) first opens the step-up dialog. The dialog calls `/auth/step-up/start` and `/verify`; the grant goes in `X-Step-Up`. The grant is used once and kept only in memory.
Idempotency	Requests that create something (an order, a payment, a bid, a dispute, a listing) carry an `Idempotency-Key` chosen once per user action and kept across the retry after a refresh. The server's use of it is I-51.
Errors (DC-3)	Problem documents become `ApiError` with status, type, correlation id, field errors and `Retry-After`. Each maps to one message (table below).

Checking for another tab's refresh. The first version of the client recorded each refresh time in `localStorage`. A tab that saw a time later than its own failed request skipped the refresh. The browser test showed that this was not enough: in 5 of 10 runs, three tabs racing after an expiry produced two refreshes. The reason is that `localStorage` writes reach other tabs with a delay, so a tab may not yet see the time another tab wrote. The second refresh was harmless (Unit 1 rotates the token again), but it was not the behaviour Unit 1 designed for. The client now uses the `localStorage` time only as a quick hint. Inside the lock, before refreshing, it sends one `GET /me`. The browser attaches whatever cookies it currently holds, and the cookie store is shared by all tabs at once. If `/me` succeeds, another tab has refreshed and the request is simply retried. With this change, 30 racing rounds over 6 runs gave one refresh each. The cost is one extra request per tab every 15 minutes (CP-01).

Errors and messages. Messages are generic, as SEC-INP-06 requires, and come from the catalogue:

Response	Shown	Then
network failure	"We could not reach TrustBazaar . . . "	Queries retry on reconnect; a state change is never retried automatically.
401 (after refresh)	"You have been signed out . . . "	Cache cleared; sign-in page, returning to the same path afterwards.
403 step-up	". . . enter the code we sent . . . "	The step-up dialog.
403 other, 404	Not allowed; no longer exists	No retry.
409	"This changed while you were looking at it . . . "	The screen's queries are refetched, so the user sees the new state before trying again.
422	Field errors next to their fields	Focus moves to the first field in error.
429	"Try again in {seconds} seconds"	The button stays disabled for `Retry-After`.
5xx	". . . quote reference {ref}"	The correlation id, so support can find the log entry.

Sent once. Every button that changes state is disabled from the click until the answer, and the request carries its idempotency key. A double click, a retry after a refresh, or a request repeated after a lost response therefore cannot pay twice or bid twice. The database rules of Units 2 and 3 stay the final barrier.

4.3 Server state

What the server knows (listings, orders, threads, the account) is held in TanStack Query's cache and nowhere else. A screen reads through a query hook and changes things through a mutation that invalidates the affected queries. Query keys follow the API paths, for example `['orders', id]` and `['auctions', id, 'state']`. Freshness depends on how fast the data changes and how much a stale view costs:

Data	Stale after	Refreshed also
Order, escrow, handover, game handover, dispute	0	On focus, on reconnect, on every notification about that order.
Auction state	live	The live feed (Section 4.4); `/state` on open, on reconnect and every 60 s.
Search results, listings	30 s	On focus.
Categories, areas, games, meetup points	1 hour	—
Own account (`/me`), KYC status	0	After sign-in, step-up and KYC submission.

After sign-out or a `401`, the whole cache is cleared, so one user's data never shows to the next user of a shared phone.

4.4 The live connection

Each tab opens one Socket.IO connection to `/socket.io` after sign-in. Unit 1 authenticates it with the session cookie when it opens (SEC-SES-07). The server places it in the user's room. The client joins an auction room while an auction page is open.

Event	From	Client handling
`auction.state`	Unit 3	`AuctionFeed.apply`: whole state with a sequence number; an older or repeated one is dropped. Its `serverTime` feeds the clock.
`auction.closed`	Unit 3	Final result; the panel shows sold or unsold, and the winner is taken to the order.
`auction.outbid`	Unit 3	A notice to the previous leader, and a refetch of `my-status`.
`chat.message`	Unit 6	`ChatThread.receive`: appended if it is the next number; our own message is recognised by its client id; a gap triggers `GET /threads/{id}/messages?after=`.
`notification.new`	Unit 6	Unread count; invalidates the queries of the object it names (I-55).

Reconnection (IF-COM-02). Socket.IO reconnects with a growing delay (DV-57). Live messages may have been missed in between, so after a reconnection the client refetches the state of every open auction, resyncs every open thread from its last sequence number, and invalidates the order queries. Phones that slept are handled the same way, because the connection drops. If a reconnection is refused because the access token expired, the client refreshes through `api.js` and connects again.

Guests do not get the live connection (Unit 3); the auction page polls `/state` every DV-19 and shows "Sign in for live updates".

Ordering, tested. The auction feed keeps only newer states in any arrival order. In the chat test, 200 messages arrive shuffled with a fifth of them lost; after resyncs the thread is complete and in order (Appendix B.1).

4.5 The server clock and countdowns

Auction ends, payment windows, takeover windows and dispute deadlines are counted down on the server's clock, because a phone's clock can be minutes wrong (FR-AUC-01). `ServerClock`

(`lib/time.js`) estimates the difference from the `serverTime` field:

- A response to a request sent at s and received at r, carrying server time T, bounds the offset: it lies between $T - r$ and $T - s$.
- A live message gives only the lower bound $T - r$, because it may have been delayed.
- The estimate is the middle of the intersection of the last 8 samples (DV-55), or its lower bound if no response has given an upper bound. A delayed message therefore never moves the estimate the wrong way. If the samples contradict each other, the device clock has jumped, and only the newest sample is kept.

One timer per tab ticks once a second and updates every visible countdown. Countdowns always show the time remaining according to the server's clock, so a bid placed in the last seconds is judged by the same clock the user saw. The server alone decides whether a bid arrived in time (Unit 3), and the screen says so near the end: "The auction ends on the server's clock; a bid sent in the last second may arrive too late."

5 Screens

Each screen is listed with what it must show, what it calls, and the rules the SRS or an earlier unit set for it. Shared components keep the rules in one place: `<Money>`, `<DateTime>` and `<Deadline>` format through `lib/` (Section 6); `<ConfirmDialog>` shows an action's consequence (IF-UI-10); `<SimulatedBanner>` appears on every payment and courier screen (NFR-CMP-05); `<StepUpDialog>` asks for the OTP.

5.1 Account and sign-in (IF-UI-01)

Screen	Calls (Unit 1)	Rules shown
Register	`/auth/register`, `/verify`, `/resend`	Age confirmation and the privacy notice before submit (FR-PRV-01); OTP resend disabled for `Retry-After`.
Sign in	`/auth/signin`, `/signin/mfa`	One error text for every failure (Unit 1 does not reveal which accounts exist); the growing delay shown as a countdown.
Reset	`/auth/password/forgot`, `/reset`	The same answer whether or not the account exists.
Security	`/auth/sessions`, `DELETE /auth/sessions/{id}`, `/end-others`, MFA, password and contact changes	Sessions with device, area and last use; "this device" marked. Changes use step-up. After a password change, other sessions are shown as ended.
Privacy	`/me/consents`, `/me/data-export`, `/me/close`	What closing removes and keeps (FR-ACC-11) before the step-up; the export link's 7 days (DV-49).

5.2 KYC wizard (IF-UI-02, NFR-USE-02)

The wizard is `kyc.html`, the only document allowed the camera. Before any capture it states what is needed: an NID or passport, both sides, a live selfie, good light, and no glare. Then it guides each capture with `getUserMedia` and shows the image for retaking. The liveness step shows the random action from `/kyc/liveness-challenge`. The status (`/kyc/status`) is shown at the top of every step, with the rejection reason when there is one. Images are sent once, at the end, to `/kyc/submissions`. They are kept only in memory until then, never in browser storage. Files already uploaded are shown through Unit 1's signed links only. When the camera is refused, the

wizard explains how to allow it and offers a file upload for the documents, though not for the selfie (FR-KYC-02).

5.3 Listings, search and the auction panel (IF-UI-03, NFR-USE-01)

Create and edit (`/sell`): category from `/categories`, where prohibited ones show their policy (FR-LST-10); photos uploaded one by one with progress, up to CP-33; the area chosen on a list, or found with the Geolocation API after a button press, with the list as the fallback (IF-SW-08). Before publishing, the screen shows the commission that will be deducted from the payout (CP-25, FR-MON-01). It also states FR-LST-07's limit: after the first bid or order, only clarifications can be added, and an auction's starting price, reserve and duration never change after publishing. A listing sent to review says so.

Search (`/search`): filters in the URL, so a search can be shared and the back button works; cursor pages appended with a "Show more" button rather than endless scrolling, so the footer and its help links stay reachable; distance shown in whole kilometres (Unit 4).

The auction panel on the listing page:

- current price, bid count, masked leader, reserve met or not, and the countdown on the server's clock;
- the minimum next bid (price plus the increment, CP-08), and an input parsed by `parseTaka`, which accepts Bangla digits. An amount below the minimum is explained before sending;
- "Your maximum" and "You are leading", from `my-status`, shown only to the bidder; a proxy bid explained in one sentence (`confirm.proxy_bid`);
- the rules (increment, proxy bidding, reserve, anti-sniping) in a panel that opens in place, so the bidder never leaves the page (NFR-USE-01);
- the masked bid history (FR-AUC-13);
- after "Place bid": the confirmation (`confirm.place_bid`), then `POST /auctions/{id}/bids` with an idempotency key. If the price moved meanwhile, the answer is shown with the new minimum. Live updates continue while the dialog is open, and the dialog shows the new price if it changes.

Price changes are announced to screen readers in a polite live region at most every 5 seconds, plus at once when the user is outbid. The countdown itself is not announced every second; it is announced at 5 minutes and 1 minute.

5.4 Checkout and payment (IF-UI-04)

A fixed-price checkout starts with `POST /orders`, which reserves the item (Unit 2); an auction win already has its order. The checkout shows one total, with the item price and the delivery fee beneath it, the fulfilment method, the payment method and the fee disclosure. The fee is the platform's commission, paid by the seller; the buyer pays the total shown and nothing more. Each fulfilment option shows its fee from the binding quote (Unit 4, DV-23), and the quote's expiry as a countdown. "Pay" opens `confirm.buy_now` and calls `POST /orders/{id}/payments` with an idempotency key.

The answer is the provider's page. The client follows it only if its host is on the list of providers: bKash, Nagad and Stripe sandboxes, and the mock provider (Section 8). After the provider, `/payments/return/:provider` shows only the order's status. It never treats the return as payment (FR-PAY-02). It polls `GET /orders/{id}` every 2 seconds for up to 30 seconds, and then says "We are still waiting for the payment provider. We will notify you" (NFR-PERF-06 expects 5 seconds). The simulated-payments banner is on every payment screen.

5.5 The order view (IF-UI-05, NFR-USE-07)

One screen for buyer and seller, built from `GET /orders/{id}` and the unit tabs. From top to bottom:

1. **Where the money is**, in one sentence from `describeMoney` (Section 6.4).
2. **Every deadline that applies**, each as a time and a countdown (NFR-USE-08): payment window, fulfilment deadline, meetup time, auto-release or the end of the protection period, dispute window, and the seller's response deadline in a dispute.
3. **What can be done now**, and only that. Each action shows its consequence first: confirm receipt (`confirm.confirm_receipt`), open a dispute, ask to cancel; for the seller, cancel (`confirm.seller_cancel`, with the strike). Only actions the server's state allows are offered, and the server checks again.
4. **Fulfilment**: the handover tab or the shipment tab (Unit 4), or the game-account tab (Unit 5).
5. **Chat** with the other party (Unit 6) and, after the order closes, the rating (Unit 7).

Handover code (Unit 4). The buyer sees the code with its instruction, `confirm.share_handover_code`: "Give this code to the seller only after you have checked the item." The code is large, readable aloud digit by digit, and hidden behind a "Show code" button, so it does not show on a screen held up at a meetup. The seller's entry screen says "Hand over the item only after the buyer gives you the code" and shows the attempts left before the lock. Meetup proposals list the curated points with a map link. Map links are offered only for these public points, never for a user's address (Unit 4). A shipment shows its status history and the "simulated" banner.

5.6 Game accounts (IF-UI-06, NFR-USE-03)

Seller, before sale: game details, then the ownership-code screen. The code is shown with where to place it in that game (from `/games`) and its expiry countdown; then the screenshot upload, and the proof's status.

Buyer, before purchase: the disclosure (FR-GAM-05), acknowledged by version. The checkout first checks that this browser has Web Crypto and IndexedDB (Unit 5, AS-5). If not, it says the purchase needs a browser that supports them and stops. It also tells the buyer: "Use this same browser on this device to open the credentials. If you clear its data or change device, you can ask the seller to send them again once."

The two credential screens are `secure.html` (Section 8):

- **Deliver** (seller): the buyer's key fingerprint, in Unit 5's five groups of four, to compare if the parties wish; the credential fields with autocomplete and spell-checking off; then `sealed.js` seals the credentials in the page and sends only the ciphertext. Nothing typed is kept after sending.
- **Credentials** (buyer): opens the ciphertext with the key in IndexedDB, shows the credentials, and next to them the takeover checklist. Each step is confirmed by its own button. The remaining takeover window (CP-23) is shown as a countdown, and the date the protection period ends beside it (NFR-USE-03). "The credentials do not work" is offered until the recovery e-mail step is confirmed (FR-GAM-14).

On the order view, early release asks for step-up and states `confirm.release_early`.

5.7 Chat (IF-UI-07)

- Text is rendered as text only (SEC-INP-03). A web address in a message is shown as plain text, not a link (Unit 6).

- A message flagged by Unit 6 shows its warning to both parties, under the message: off-platform payment, contact details, or login details. For login details, the warning points the seller to the sealed handover.
- Each message gets a random client id when it is typed, and keeps it for its resends; it shows "sending" until the server's copy arrives (`ChatThread`).
- Block and report are on every thread. Images are shown through signed links, with their size limited in the layout.

5.8 Disputes and ratings (Unit 7)

The dispute flow starts from the order view, and only the types allowed now are offered; the server answers that through Unit 7. The flow takes a type, a description of at least 20 characters, and up to 10 files (CP-33), and shows `confirm.open_dispute`. The dispute tab then shows the status, the seller's response deadline and the decision target as countdowns, both parties' statements, and, after the decision, the outcome and its reason. Withdrawing shows `confirm.withdraw_dispute` with the new auto-release time. The rating form appears after the order closes, with the end of the rating window and of the edit period (CP-39).

5.9 Seller dashboard (IF-UI-08)

Listings by state, orders needing action (with their deadlines first), the payout account and its check status, payouts and statements; for Pro Sellers, sales analytics (sales, views to orders, average price, by month). Figures come from the server; the dashboard computes nothing about money.

5.10 Staff console (IF-UI-09)

`staff.html`, signed in with staff accounts only, with MFA (Unit 1).

- **Queues**: the eight queues of FR-OPS-01 (IF-UI-09 names five; the console shows all eight, Unit 7). Each queue is ordered oldest first, with overdue items on top and each item's age.
- **Claim**: a case opens only after a claim; a conflict refusal (FR-OPS-06) is shown as its message.
- **Case file**: personal data is masked by default (SEC-ACC-05); "Show" asks for a reason, which the server logs. KYC images and evidence open through signed links that expire (CP-31). The chat thread of the case is shown only when the case holds it.
- **Actions**: only the actions of FR-OPS-02, each with a required reason. A dispute decision shows the outcome preview (the exact split from Unit 2) and, at CP-30, "This decision needs a second operator's approval".
- **Approvals**: the open four-eyes proposals for the staff member's role, never their own.
- **Administrators**: staff accounts; the parameter editor, which shows every broken rule at once (Unit 7's `config_violations`); emergency switches with four-eyes; reference data (games, areas, meetup points, screening and detection terms); the audit log, read-only.

The console has no "edit user", "edit order" or "edit payment" screen, because none exists in the API (FR-OPS-02). The console shows staff sessions' shorter idle limit (CP-03) as a warning a minute before sign-out.

5.11 Consequences before confirmation (IF-UI-10)

Every action that moves money, ends a right or cannot be undone opens `<ConfirmDialog>` with its consequence message, filled with the real amounts and times. The confirm button names the action and its amount ("Release Tk 12,550 to the seller") rather than saying "OK". The reference

catalogue has the nine messages these actions need: confirm receipt, early release, opening and withdrawing a dispute, seller cancellation, bids, proxy bids, buying and the handover code. A test checks that each exists in both languages with the same placeholders.

6 Language, Money and Time

6.1 Bangla and English (NFR-USE-04)

- **Choice.** A switch in the header of every page, at any time. A signed-in user's choice is saved in their notification preferences (Unit 6), so notices use the same language. A guest's choice is kept in `localStorage`. With no choice yet, the app uses Bangla if the browser prefers Bangla, and English otherwise (DV-58). `<html lang>` follows the choice, so screen readers pronounce correctly; text from the other language inside a page (a user's listing title, say) is not marked up, because its language is not known.
- **Catalogues.** One file per feature and language, loaded with the feature's code, with the same keys in both languages (tested). Messages are plain text with `{name}` placeholders, filled with text that is already formatted (money, times). No message contains markup (tested), so a translation can never inject HTML. In development, a missing key or placeholder throws; in production it shows the key, which the catalogue test makes impossible to ship.
- **Server text.** Category names, games and areas arrive in both languages from the API. Notification titles arrive in the user's language (Unit 6). Error messages are always the client's own, chosen by status (Section 4.2).
- **Bangla text** in the reference catalogue was written for this design. The team should review it before release, and a native speaker should review the full catalogue.
- **Font.** Noto Sans Bengali, self-hosted as WOFF2 with `font-display: swap`, and preloaded only when the language is Bangla. The Latin text uses the system font.

6.2 Money

Amounts arrive as integer poisha and stay integers: `formatMoney` works on `BigInt`, so an amount is never rounded on its way to the screen (tested above 2^{53} poisha). The display uses the taka sign before the amount and lakh grouping (12,34,567) in both languages, with Bangla digits in Bangla. A single price drops ".00"; the lines of a breakdown and a receipt always show two decimals, so they line up (DV-59). Input goes through `parseTaka`. It accepts Bangla or ASCII digits, group commas in either the lakh or the Western style, an optional taka sign, "Tk" or "BDT", and up to two decimals. It never passes through a floating-point number, so 0.1 taka is exactly 10 poisha. Every refusal has a reason the form can show (empty, wrong format, too many decimals, too large); limits such as the minimum bid are the server's, and the form shows them before sending.

6.3 Dates, times and deadlines (NFR-USE-08)

Every time is formatted in `Asia/Dhaka`, whatever the device's zone, with weekday, date, month, year and time: "Tue 20 Oct 2026, 6:00 pm". The tests pass with the device in four other zones, and Chromium set to New York produces the same text as Node. The day period in Bangla is set by the module (*pūrbāhna*, *aparāhna*), because browsers' locale data differ here: Node's prints "PM" inside Bangla text. Every deadline is shown as a time and a countdown: "Tue 20 Oct 2026, 6:00 pm (in 2 days 1 h)". Countdowns use days and hours above a day, hours and minutes above an hour, and minutes and seconds below; they never show 0:00 before the end. In the last 5 minutes they turn urgent, shown by both colour and text.

6.4 The money state in plain language (NFR-USE-07)

`describeMoney(order, role)` chooses one of nine sentences for each role from the order's status, escrow state, release policy and deadlines, and `renderMoney` fills it. The SRS's example is reproduced word for word by the test:

> "Your payment is held by TrustBazaar. The seller is paid when you confirm receipt, or automatically on Tue 20 Oct 2026, 6:00 pm unless you open a dispute."

State	Buyer sees (the seller has a matching sentence)
awaiting payment	Nothing has been charged yet. Pay by {due} to keep this order.
held, before delivery	Your payment of {amount} is held by TrustBazaar. The seller must hand the item over by {due}, or you get a full refund.
held, after delivery	The SRS sentence above.
held, game protection	Your payment is held until the protection period ends on {at}. If the seller takes the account back before then, open a dispute.
frozen (dispute)	Your payment of {amount} is frozen while TrustBazaar reviews your dispute. Nobody is paid until it is decided.
released, refunded, partially refunded	Who got how much, and when.

The sentence is chosen from the server's fields only; the client never infers a state the server has not reported. An unknown escrow state throws instead of showing a guess (tested).

7 Accessibility and Small Screens

Small screens (NFR-USE-05). The layout is designed for small screens first, as one column, and is checked at 320 CSS pixels, the WCAG reflow width, which is narrower than the SRS's 360. Tables in the console and the seller dashboard become stacked cards below 600 pixels. Long unbroken strings (order references, fingerprints, links in chat) may wrap anywhere. The planned end-to-end run visits every route at 360 by 740 and fails if the page scrolls sideways (Section 12).

WCAG 2.2 AA (NFR-USE-06). The choices that matter most for this site:

- native elements first: buttons, links, form controls, `<dialog>` for confirmations, with focus moved into the dialog and back when it closes;
- every control reachable and usable by keyboard; a visible focus ring of at least 2 pixels and 3:1 contrast;
- targets of at least 24 by 24 pixels (2.5.8), and 44 for the main actions (Pay, Place bid, Confirm receipt);
- text contrast of at least 4.5:1 from the design tokens; no state shown by colour alone (escrow, urgency and warnings always have text or an icon with a label);
- every field with a visible label; errors next to their field and in a summary at the top, linked to the field, with focus moved to the summary;
- live regions only where a change matters: bids and outbid notices (polite, at most every 5 seconds), chat messages, step-up and errors. Countdowns are `role="timer"` and are not read out every second; they are announced at 5 minutes and 1 minute;
- reduced motion respected; no content that flashes;
- time limits (2.2.1): auctions are a real-time event, which WCAG exempts. A checkout quote or a payment window that expires can be started again from the same screen.

The planned end-to-end run checks every route with axe-core and fails on any serious or critical issue (Section 12). Manual checks with a screen reader (TalkBack on Android, NVDA on Windows) cover the bid, checkout, order and handover-code flows.

8 Security

8.1 Security headers per document

The web container sets the headers of the four documents; the backend sets its own on API responses (Unit 1, DC-11). The reference configuration is `docs/design/web/container/` (Appendix B). It was run in nginx 1.24 and checked from Chromium (Section 12).

Header	Value and reason
CSP, user app and console	`default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data: blob:; connect-src 'self'; object-src 'none'; frame-src 'none'; base-uri 'none'; form-action 'self'; frame-ancestors 'none'; require-trusted-types-for 'script'; trusted-types 'none'`. No inline script, no `eval`, no third-party origin (SEC-WEB-01). The payment providers are reached by full-page redirect, not framed or scripted, so they need no place in the policy and SEC-WEB-01's exception is not used.
CSP, KYC	The same, with `media-src 'self' blob:` for the camera preview.
CSP, credentials	`default-src 'none'` and only `'self'` for scripts, styles, images, fonts and connections; `form-action 'none'` (SEC-GAM-03).
Trusted Types	With `trusted-types 'none'`, any attempt to put an HTML string into the page through `innerHTML` or a similar sink throws. That backs SEC-INP-03 in the browser itself. React renders text through DOM methods and works under it (tested); `dangerouslySetInnerHTML` would fail, and is also banned by ESLint.
Permissions-Policy	Camera only in `kyc.html`; geolocation only in the user app; microphone, payment, USB, serial, Bluetooth and the motion sensors nowhere (SEC-WEB-02).
Cache-Control	HTML `no-cache`, so a new release reaches every user on their next page load; the console, KYC and credential pages `no-store`; hashed assets cached for a year.
Others	`X-Content-Type-Options: nosniff`; `Referrer-Policy: same-origin` (`no-referrer` on the console and credentials); `Cross-Origin-Opener-Policy` and `-Resource-Policy: same-origin`. HSTS is set by the TLS front (Unit 9).

Development. Vite's development server injects an inline script for hot reloading, so these policies apply to the production build only. The end-to-end tests run against the production build in the container, never the development server.

Geolocation per document. SEC-WEB-02 allows the camera and geolocation "only on the pages that need them". A single-page app cannot narrow a permission below its document. The camera has a document of its own, as the requirement intends. Geolocation is allowed in the user app, where search and listing creation use it, and is requested only when the user presses "Use my location". An SRS clarification states this (I-58).

8.2 Other client-side rules

- **Text only** (SEC-INP-03). User text is rendered as React text. ESLint forbids `dangerouslySetInnerHTML` and direct `innerHTML`, and Trusted Types enforces the same rule in the browser. Links in user text are not made clickable.
- **Leaving the site.** The client follows a payment redirect only to an exact, configured provider host over HTTPS, or to a same-origin path for the mock provider (`checkProviderRedirect`, tested against look-alike hosts, other ports, credentials in the URL, `javascript:` and protocol-relative URLs). The host list comes from Unit 2's adapter settings at build time.
- **Storage.** Browser storage holds only: the language choice for guests, the time of the last token refresh, and, in IndexedDB, the buyer's non-extractable game-account key (Unit 5). ESLint allows `localStorage` only in the modules that keep those. No personal data, no draft containing personal data, and no API response is stored.

- **Masking** (SEC-ACC-05). The console shows the masked values the API returns. "Show" sends a reason, and only the server's answer, which is logged, contains the full value.
- **Dependencies.** Exact versions in the lock file; `npm audit` in CI; no script or style from another origin, so SEC-WEB-04's integrity hashes have nothing to apply to.
- **Simulated services** (NFR-CMP-05). `<SimulatedBanner>` on checkout, payment return, shipment and payout screens, in both languages.

9 Performance

NFR-PERF-07 asks for the largest contentful paint within 2.5 seconds on the home, listing and checkout pages, on 4G and a mid-range phone.

- **Code splitting.** Each route is a lazy chunk. The first load of the user app carries React, the router, the query cache, `lib/` and the page's own chunk; the live connection is loaded after the first paint. The budget is 170 KB of compressed JavaScript for the first load of these three pages (DV-53), to be checked in CI from the build's chunk sizes.
- **The largest paint** on a listing page is its first photo. It uses Unit 3's 1600-pixel image with the 400-pixel thumbnail as `srcset` alternatives, explicit width and height (no layout shift), and `fetchpriority="high"`. Other photos and search thumbnails load lazily.
- **Data.** The listing page renders from one request (`GET /listings/{id}`, which includes the auction state); search results come from one request with cursor pages.
- **Measurement** (planned). Lighthouse runs in CI against the production build, with its mobile profile (simulated 4G and a slowed processor), on the three pages, and fails above 2.5 seconds.

The live paths are measured end to end by Units 3 and 6 (NFR-PERF-03, 04). The client adds one state update and a render, which takes milliseconds.

10 Build and the Web Container

- **Vite.** Four HTML entries (`build.rollupOptions.input`); hashed file names under `/assets/`; target ES2020, matching the supported browsers: the last two versions of Chrome, Edge, Firefox and Safari, and Chrome and Samsung Internet on Android. The development server proxies `/api` and `/socket.io` (with WebSocket) to the backend.
- **Configuration.** Only build-time values that are not secret: the provider host list and the feature flags known at build time. Parameters (fees, windows, limits) are never built in. They come from the API, which reads Unit 7's current version (NFR-MNT-06).
- **Dockerfile.** Two stages: `node:20-alpine` runs `npm ci` and `npm run build`; `nginx:1.27-alpine` serves `dist/` with the reference configuration and proxies to `backend:3000`. This is the web container of NFR-MNT-01, and it replaces the current Dockerfile, which runs the development server. Developers still use `npm run dev` outside Docker.
- **Health.** `GET /healthz` answers from nginx without touching the backend, for Compose's health check (Unit 9).

11 Parameters

SRS parameters shown or used by the client: every deadline parameter through the server's values (CP-10, CP-11, CP-17 to CP-24, CP-39); CP-01 and CP-03 (session lengths, for the refresh and the console's warning); CP-08 (shown as the minimum next bid, from the server); CP-25 (commission shown to sellers); CP-31 (signed links); CP-33 (upload limits checked before sending);

CP-35 (limits answered by `429`). None is built into the client (NFR-MNT-06).

Design values introduced (continuing DV-1 to DV-52):

ID	Value	Default	Reason
DV-53	First-load JavaScript, home, listing and checkout	at most 170 KB compressed	NFR-PERF-07 on 4G.
DV-54	Countdowns	tick 1 s; urgent in the last 5 min; announced at 5 and 1 min	Readable without flooding screen readers.
DV-55	Server clock	last 8 samples; `/state` again every 60 s on an auction page	Corrects drift and sleeping devices.
DV-56	Cache freshness	orders and money 0; search 30 s; reference data 1 h	A stale money view costs trust.
DV-57	Live reconnection	0.5 s doubling to 10 s, with random spread	Many phones reconnecting at once do not arrive together.
DV-58	Default language	Bangla if the browser prefers it, otherwise English	Most users are in Bangladesh.
DV-59	Money decimals	dropped when zero for single prices; always two in breakdowns	Clean prices; aligned sums.
DV-60	After the payment provider	ask every 2 s for up to 30 s	NFR-PERF-06 is 5 s; the rest covers slow callbacks.

12 Testing

Level	Cases
Reference modules (done)	46 tests with Node's test runner (`npm run test:unit`). API client: CSRF on state changes only; one refresh and one retry with the same idempotency key; ten failing requests, one refresh; another tab's refresh reused, also when its `localStorage` mark is not yet visible; `409` from refresh retried; a refused refresh, and a `401` after a successful one, sign the user out; problem documents with correlation id and `Retry-After`; network failures and aborts; step-up grants; the staff console's own cookie and refresh path. Money: lakh grouping, Bangla digits, exactness above 2^{53}, refusal of fractional poisha; parsing of 10 valid and 10 invalid inputs, each with its reason. Time: Dhaka time with the date, the change of date at Dhaka midnight, countdowns, deadlines; the server clock's bounds, delayed messages, clock jumps. Catalogues: same keys and placeholders in both languages, Bangla script and digits, no markup, every consequence message and every money-state message present; the SRS sentence of NFR-USE-07 word for word, and every escrow state for both roles. Live ordering: auction states in any order; chat duplicates, gaps, resends, and 200 shuffled messages with a fifth lost. Redirects: providers, same-origin paths, and nine kinds of unsafe target. The time tests pass with the device set to four other zones.
Browser (done)	12 checks in Chromium 141 against a fake Unit 1 with real cookies (`npm run test:browser`): the session cookies cannot be read by the page and the CSRF cookie can; 15 requests from 3 tabs after an expiry all succeed with exactly one refresh, and so do five further rounds; the refresh carries the CSRF token; no `Authorization` header and nothing but the refresh time in `localStorage`; a refused refresh signs the tab out; Dhaka times and money text in a browser set to New York equal Node's. This test found the race described in Section 4.2: it failed in 5 of 10 runs before the fix and passed 6 runs of 5 rounds after it.

Level	Cases
Web container (done)	25 checks with nginx 1.24 running the reference configuration and Chromium loading four documents built like the application (`npm run test:container`): `nginx -t`; ten paths, including deep links, each served by its document; cache, referrer, framing and `nosniff` headers; assets cached for a year and served as JavaScript; API responses without a page policy; a missing asset gives 404; the health check. In each document: React 18 renders and handles events; style props and the stylesheet work; an inline script never runs and an inline style block is refused; `innerHTML` with a string throws (Trusted Types); `eval` is refused; each refusal is reported; only the KYC document may use the camera and only the user app may ask for the location; the API and the WebSocket pass through the proxy on the same origin.
Components (planned)	Vitest and Testing Library: `<Money>`, `<Deadline>`, `<Countdown>` (announcements), `<ConfirmDialog>` (focus moved in and back, Escape), `<StepUpDialog>`, the bid form (minimum, Bangla digits, price moved meanwhile).
End to end (planned)	Playwright against the Compose system with the mock providers (Unit 9): register, KYC with Chromium's fake camera, list, buy with the mock bKash, handover code, confirm receipt; two browsers bidding live on one auction; the sealed game-account handover between two browser contexts; a dispute decided in the console, including four-eyes; sign-out on one tab ends the others. On every route: no sideways scroll at 360 by 740 and at 320, no axe-core issue rated serious or critical, and no CSP or Trusted Types violation.
Performance (planned)	Lighthouse CI on home, listing and checkout with the mobile profile; the bundle budget DV-53 from the build output.

13 Notes for Integration

I-51 Unit 1. (a) Staff sessions use their own cookies, for example `__Host-tb_sat`, `__Host-tb_scsrf` and `__Secure-tb_srt` on `/api/v1/staff/auth/refresh`, with `POST /staff/auth/refresh` and `GET /staff/me`. Then a staff session and a user session in one browser do not overwrite each other. (b) Honour `Idempotency-Key` on creating requests: keep the key with the principal and the response for 24 hours in Redis; the same key with a different body is `422`. (c) `GET /me` is the refresh probe: it must stay cheap and must not be rate-limited below a few calls a minute per session. (d) The Socket.IO handshake answers an expired access token with a distinct error, so the client refreshes and reconnects.

I-52 Unit 2. Idempotency keys on `POST /orders` and `/orders/{id}/payments`. `GET /orders/{id}` returns every field `describeMoney` reads: status, escrow state, release policy, total, refunded, the deadlines, the delivery event time and the close time; and the item price and delivery fee for the checkout's confirmation. The providers' page hosts are part of the adapter settings, for the build's redirect list.

I-53 Unit 3. Idempotency keys on `POST /auctions/{id}/bids` and `POST /listings`. A bid's answer carries the new state with `seq` and `serverTime`. The state includes the minimum next bid, so the client never computes the increment. `GET /listings/{id}` embeds the auction state, so the page renders from one request.

I-54 Units 4 and 5. The handover answer includes the code attempts left. The credential screens are routes under `/secure/orders/{id}/`, served by `secure.html`, which imports `sealed.js` unchanged; Unit 5's links and notices point there.

I-55 Unit 6. `notification.new` names the object it concerns (for example the order id), so the client refreshes exactly that data. The language in the notification preferences is the client's language choice.

I-56 Unit 7. The dispute form needs the types that may be opened now: `GET /orders/{id}/`

`dispute-types` answers from the owning units' checks. It is a new endpoint, used by the dispute flow of Section 5.

I-57 Unit 9. The web container is the two-stage Dockerfile with the reference nginx configuration, and its health check is `/healthz`. HSTS and TLS are set at the front. The end-to-end, accessibility and performance runs use the Compose system.

I-58 SRS clarifications proposed:

- SEC-WEB-02: "page" means a document. The camera is allowed only in the KYC document, and geolocation in the user app, requested only on the user's action;
- IF-UI-09 lists five operator queues and FR-OPS-01 eight; the console shows the eight;
- NFR-USE-05: the design is tested at 320 pixels as well, the WCAG reflow width;
- add DV-53 to DV-60 to the SRS parameter table.

A Traceability

Requirement	Design element	Section
IF-UI-01	Register, sign in, reset, security screen with sessions	5
IF-UI-02	KYC wizard in `kyc.html`; status on every step	5
IF-UI-03	Listing editor, search, listing page with the auction panel	5
IF-UI-04	Checkout with one total, fees and the simulated banner	5
IF-UI-05	Order view: money sentence, deadlines, actions, handover code	5.5
IF-UI-06	Ownership proof; `secure.html` credential screens with countdowns	5
IF-UI-07	Chat as text, warnings on flagged messages, block and report	5
IF-UI-08	Seller dashboard	5
IF-UI-09	`staff.html`: the eight queues, case files, admin views	5.10
IF-UI-10	`<ConfirmDialog>` with the consequence messages (tested)	5
IF-COM-01	Same origin over HTTPS and WSS; proxy tested	4
IF-COM-02	Resync after reconnection; `AuctionFeed`, `ChatThread` (tested)	4.4
IF-SW-07	`sealed.js` in `secure.html`; Web Crypto check before purchase	5
IF-SW-08	Location on the user's action, with the area list as fallback	5
FR-AUC-01	Countdowns on `ServerClock` (tested)	4.5
FR-PAY-02	The return page shows status only and polls the order	5
NFR-USE-01	Auction rules panel opened in place	5
NFR-USE-02	Requirements before capture; status on every step	5
NFR-USE-03	Takeover window countdown and protection end date	5
NFR-USE-04	Two catalogues with the same keys (tested); language switch	6
NFR-USE-05	Mobile-first layout, checked at 360 and 320 (planned run)	7
NFR-USE-06	WCAG 2.2 AA choices; axe-core run (planned)	7
NFR-USE-07	`describeMoney`; the SRS sentence (tested)	6.4
NFR-USE-08	`formatDateTime`, `formatDeadline` in Dhaka time (tested)	6
NFR-PERF-07	Code splitting, image priority, budget DV-53, Lighthouse (planned)	9
SEC-ACC-01	Route rules are guidance only; the server authorises	3.3
SEC-ACC-05	Masked values in the console; reasons for showing	5.10
SEC-SES-03	No token in reach of scripts (tested in Chromium)	4.2
SEC-SES-04	`X-CSRF-Token` on state changes (tested)	4.2
SEC-SES-07	One authenticated socket per tab; rooms by the server	4.4
SEC-INP-03	React text, ESLint rule, Trusted Types (tested)	8
SEC-WEB-01	CSP per document, no inline script (tested)	8
SEC-WEB-02	Permissions-Policy per document; nosniff; referrer (tested)	8
SEC-WEB-03	Same origin; no cross-origin calls from the client	4
SEC-WEB-04	No third-party script or style	8
SEC-GAM-03	`secure.html`: strictest policy, no-store (tested)	8
NFR-MNT-01	Two-stage web container with nginx	10
NFR-MNT-06	No parameter built into the client	10
NFR-MNT-07	ESLint with the security and import rules	8

Requirement	Design element	Section
NFR-CMP-05	<SimulatedBanner>; catalogue messages	8

B Reference Code and Tests

The reference code is in `docs/design/web/`: `src/` (the modules the application imports unchanged), `src/messages/` (the two catalogues), `container/` (the nginx configuration and the header files of the four documents), and `test/`. `npm install`, then `sh test/run_all.sh`, runs every test; the container test needs nginx on the path. In the listings below, Bangla characters are shown as their code points (<U+09F3> is the taka sign); the files contain the characters themselves. The Bangla catalogue is in the repository only.

B.1 Test results

```
Unit tests (node --test)
PASS  cookie reading
PASS  SEC-SES-04 state-changing requests carry the CSRF token; reads do not
PASS  SEC-SES-02 an expired token is refreshed once; the retry keeps its Idempotency-Key
PASS  ten requests failing together cause one refresh in the tab
PASS  a refresh done by another tab after our request was sent is reused
PASS  another tab refreshed but its localStorage mark is not visible yet: the probe sees the new cookies
PASS  409 from refresh (rotated by another tab a moment ago) is retried
PASS  SEC-SES-05 a refused refresh signs the user out
PASS  still 401 after a successful refresh: signed out, no second refresh
PASS  SEC-INP-06 problem documents keep their correlation id; 429 keeps Retry-After
PASS  network failures and aborts
PASS  SEC-AUTH-08 a step-up action obtains a grant first and sends it in X-Step-Up
PASS  the staff console uses its own CSRF cookie and refresh path
PASS  204 returns null
PASS  NFR-USE-04 both catalogues have the same messages
PASS  each message has the same placeholders in both languages
PASS  Bangla messages are written in Bangla, with Bangla digits
PASS  SEC-INP-03 no message contains markup
PASS  IF-UI-10 every irreversible action has a consequence message
PASS  every message the money-state function can pick exists in both catalogues
PASS  strict mode catches a missing key or variable
PASS  NFR-USE-07 the SRS example sentence
PASS  money state for every escrow state and both roles
PASS  auction: only newer states are applied, in any arrival order
PASS  auction: the state fetched after a reconnection is applied only if newer
PASS  chat: in-order messages append; duplicates are ignored
PASS  chat: a gap asks for a resync, which fills it in order
PASS  chat: our pending message is confirmed by its client id, once
PASS  chat: 200 messages shuffled and partly lost end complete and ordered after resync
PASS  lakh grouping with the taka sign first
PASS  Bangla digits in Bangla
PASS  decimals: auto drops .00, always keeps it
PASS  signs and exactness beyond floating point
PASS  non-integer amounts are refused
PASS  typed amounts become exact poisha
PASS  bad input is refused with its reason
PASS  configured providers over HTTPS, and same-origin paths, are followed
PASS  anything else is refused
PASS  NFR-USE-08 times are shown in Dhaka time with the date, whatever the device zone
PASS  the date changes at Dhaka midnight, not UTC midnight
PASS  countdowns
PASS  a deadline is a time and a countdown
PASS  server clock: HTTP samples bound the offset on both sides
PASS  server clock: a delayed live message never pulls the estimate down
PASS  server clock: a device clock jump resets to the newest sample
PASS  server clock: message samples alone give the best lower bound
tests 46 pass 46 fail 0 (Node v22.22.0)

Browser tests (Chromium): cookies, CSRF, refresh across tabs, Intl
PASS  SEC-SES-03 the page cannot read the session cookies
PASS  SEC-SES-04 the CSRF cookie is readable, so it can be copied into the header
```

```
PASS  a POST through the client passes the CSRF check
PASS  15 requests from 3 tabs with an expired token all succeed
PASS  Unit 1 "parallel tabs": exactly one refresh for 3 tabs (got 1)
PASS  the refresh request carries the CSRF token
PASS  localStorage holds only the refresh time, no token
PASS  no request carries an Authorization header
PASS  five later expiries give one refresh each (got 1, 1, 1, 1, 1)
PASS  a refused refresh signs the tab out
PASS  Dhaka date and time text in Chromium matches Node (8 strings, en and bn)
PASS  money text and parsing in Chromium match Node

12 checks passed, 0 failed (Chromium 141.0.7390.37, browser time zone America/New_York)

Web container tests (nginx and Chromium): documents, headers, policies, proxy
PASS  nginx -t accepts the configuration
PASS  every path is served by its document (10 paths, including deep links)
PASS  SEC-GAM-03 credential pages: Cache-Control no-store
PASS  credential pages: CSP starts from nothing allowed
PASS  credential pages: no referrer
PASS  user app HTML is revalidated on every load, so a new release applies at once
PASS  SEC-WEB-01 frame-ancestors 'none'
PASS  SEC-WEB-02 nosniff
PASS  hashed assets are cached for a year
PASS  scripts are served as JavaScript
PASS  API responses carry the backend's headers, not a page policy
PASS  a missing asset is 404, not the app page
PASS  the health check answers without the backend
PASS  React 18 renders and handles events under every document's policy
PASS  React style props still work (they are set through the CSSOM, which CSP allows)
PASS  the built stylesheet applies
PASS  SEC-WEB-01 an inline script never runs
PASS  an inline <style> block is refused
PASS  SEC-INP-03 Trusted Types: assigning HTML through innerHTML throws
PASS  eval and new Function are refused
PASS  each refusal is reported as a policy violation
PASS  SEC-WEB-02 only the KYC document may use the camera
PASS  only the user app may ask for the location
PASS  the API is reached on the same origin, through the proxy
PASS  the WebSocket upgrade passes the proxy and connect-src 'self'

25 checks passed, 0 failed (nginx/1.24.0, Chromium 141.0.7390.37)
```

B.2 src/api.js

```
// TrustBazaar web client - HTTP API client (Unit 8 reference module).
// Unit 1 decides the rules this client follows:
//  * tokens live only in HttpOnly cookies; this code never sees them (SEC-SES-03);
//  * every state-changing request carries X-CSRF-Token, copied from the readable __Host-tb_csrf
//    cookie (signed double submit, SEC-SES-04);
//  * an expired access token gives 401; the client refreshes once and retries once. Refreshes are
//    serialised across tabs with the Web Locks API, and a tab that waited for another tab's
//    refresh does not refresh again: inside the lock it first checks whether its cookies already
//    work (Unit 1, "Parallel tabs");
//  * errors are RFC 9457 problem documents with a correlationId (DC-3, SEC-INP-06).

const STATE_CHANGING = new Set(['POST', 'PUT', 'PATCH', 'DELETE']);
const REFRESH_MARK = 'tb.refreshedAt';     // a time, not a secret: when some tab last refreshed

export class ApiError extends Error {
  /**
   * @param {number} status 0 for network failures
   * @param {{type?: string, title?: string, correlationId?: string, errors?: unknown, retryAfter?: number}}
     p
   */
  constructor(status, p = {}) {
    super(p.title ?? `HTTP␣${status}`);
    this.status = status;
    this.type = p.type ?? (status === 0 ? 'network' : 'about:blank');
    this.correlationId = p.correlationId;
    this.errors = p.errors;
    this.retryAfter = p.retryAfter;
  }
  /** the catalogue key the screen shows for this error */
```

```
  get messageKey() {
    if (this.status === 0) return 'error.network';
    if (this.status === 401) return 'error.session_ended';
    if (this.status === 403 && this.type.endsWith('/step-up-required')) return 'error.step_up';
    if (this.status === 403) return 'error.forbidden';
    if (this.status === 404) return 'error.not_found';
    if (this.status === 409) return 'error.conflict';
    if (this.status === 429) return 'error.rate_limited';
    return 'error.generic';
  }
}

/** @param {string} cookieHeader @param {string} name */
export function readCookie(cookieHeader, name) {
  for (const part of cookieHeader.split(';')) {
    const i = part.indexOf('=');
    if (i > 0 && part.slice(0, i).trim() === name) return decodeURIComponent(part.slice(i + 1).trim());
  }
  return null;
}

/**
 * @param {object} o
 * @param {string} [o.base]
 * @param {typeof fetch} [o.fetch]
 * @param {() => string} [o.cookies]                document.cookie
 * @param {{request: (name: string, cb: () => Promise<unknown>) => Promise<unknown>}} [o.locks]  navigator.
    locks
 * @param {{getItem(k: string): string|null, setItem(k: string, v: string): void}} [o.store]  localStorage
 * @param {(action: string) => Promise<string>} [o.stepUp]  shows the OTP dialog; resolves to a grant
 * @param {() => void} [o.onSignedOut]
 * @param {string} [o.probePath]                  a cheap authenticated GET (Unit 1: GET /me)
 * @param {string} [o.csrfCookie]                 '__Host-tb_csrf'; the staff console uses its own (I-51)
 * @param {string} [o.refreshPath]                '/auth/refresh'; staff: '/staff/auth/refresh'
 * @param {() => number} [o.now]
 * @param {(ms: number) => Promise<void>} [o.sleep]
 */
export function createApi({
  base = '/api/v1', fetch: f = globalThis.fetch.bind(globalThis), cookies = () => document.cookie,
  locks = globalThis.navigator?.locks, store = globalThis.localStorage, stepUp, onSignedOut = () => {},
  probePath = '/me', csrfCookie = '__Host-tb_csrf', refreshPath = '/auth/refresh', now = () => Date.now(),
    sleep = (ms) => new Promise((r) => setTimeout(r, ms)),
} = {}) {
  /** @type {Promise<boolean> | null} one refresh at a time inside this tab */
  let inFlight = null;

  /** @param {number} failedAt when the request that got 401 was sent */
  async function refresh(failedAt) {
    if (!inFlight) {
      const run = async () => {
        // Another tab may have refreshed while we waited for the lock; its cookies are ours too.
        // The mark in localStorage is a fast hint only: tabs see each other's localStorage writes
        // with a delay, so a missing mark proves nothing. The probe asks the cookie jar itself.
        const mark = Number(store?.getItem(REFRESH_MARK + refreshPath) ?? 0);
        if (mark >= failedAt) return true;
        const probe = await f(base + probePath, { method: 'GET', credentials: 'same-origin',
          headers: { Accept: 'application/json' } }).catch(() => null);
        if (probe?.ok) return true;
        for (let attempt = 0; attempt < 2; attempt++) {
          const r = await f(base + refreshPath, { method: 'POST', credentials: 'same-origin',
            headers: { 'X-CSRF-Token': readCookie(cookies(), csrfCookie) ?? '' } });
          if (r.status === 204 || r.ok) { store?.setItem(REFRESH_MARK + refreshPath, String(now())); return
    true; }
          if (r.status === 409) { await sleep(300); continue; }   // rotated a moment ago by another tab
          return false;
        }
        return false;
      };
      inFlight = (locks ? locks.request('tb-refresh', run) : run())
        .then((ok) => /** @type {boolean} */ (ok)).finally(() => { inFlight = null; });
    }
    return inFlight;
  }

  /**
```

```
   * @param {string} method
   * @param {string} path            under /api/v1
   * @param {object} [o]
   * @param {unknown} [o.body]
   * @param {string} [o.stepUpAction]  Unit 1 step-up action; a grant is obtained first (SEC-AUTH-08)
   * @param {boolean} [o.idempotent]   send an Idempotency-Key that stays the same across retries
   * @param {AbortSignal} [o.signal]
   * @returns {Promise<any>}
   */
  async function request(method, path, { body, stepUpAction, idempotent = false, signal } = {}) {
    /** @type {Record<string, string>} */
    const headers = { Accept: 'application/json, application/problem+json' };
    if (body !== undefined) headers['Content-Type'] = 'application/json';
    if (idempotent) headers['Idempotency-Key'] = globalThis.crypto.randomUUID();
    if (stepUpAction) {
      if (!stepUp) throw new Error('stepUp handler not configured');
      headers['X-Step-Up'] = await stepUp(stepUpAction);
    }
    let refreshed = false;
    for (;;) {
      if (STATE_CHANGING.has(method)) headers['X-CSRF-Token'] = readCookie(cookies(), csrfCookie) ?? '';
      const sentAt = now();
      let res;
      try {
        res = await f(base + path, { method, headers, credentials: 'same-origin', signal,
          body: body === undefined ? undefined : JSON.stringify(body) });
      } catch (e) {
        if (/** @type {any} */ (e)?.name === 'AbortError') throw e;
        throw new ApiError(0, { title: 'network' });
      }
      if (res.status === 401) {
        if (!refreshed) {
          refreshed = true;
          if (await refresh(sentAt)) continue;    // same headers, so the same Idempotency-Key
        }
        onSignedOut();                            // refresh refused, or still 401 after it
      }
      if (res.status === 204) return null;
      const text = await res.text();
      const data = text ? JSON.parse(text) : null;
      if (res.ok) return data;
      const retry = res.headers.get('Retry-After');
      throw new ApiError(res.status, { ...(data ?? {}), retryAfter: retry ? Number(retry) : undefined });
    }
  }

  return {
    request,
    get: (/** @type {string} */ p, o = {}) => request('GET', p, o),
    post: (/** @type {string} */ p, /** @type {unknown} */ body, o = {}) => request('POST', p, { ...o, body
     }),
    put: (/** @type {string} */ p, /** @type {unknown} */ body, o = {}) => request('PUT', p, { ...o, body }),
    patch: (/** @type {string} */ p, /** @type {unknown} */ body, o = {}) => request('PATCH', p, { ...o, body
      }),
    del: (/** @type {string} */ p, o = {}) => request('DELETE', p, o),
  };
}
```

B.3 src/money.js

```
// TrustBazaar web client - money display and input (Unit 8 reference module).
// Amounts travel as integer poisha (1 taka = 100 poisha) and are never held as fractional numbers.
// Display: taka sign first, lakh grouping (12,34,567), Bangla digits in Bangla (NFR-USE-04).

const BN = '<U+09E6><U+09E7><U+09E8><U+09E9><U+09EA><U+09EB><U+09EC><U+09ED><U+09EE><U+09EF>';
const MAX_POISHA = 10n ** 13n;            // BDT 100 billion: a sanity bound only; business limits are the
     server's

/** @param {string} s @param {'en'|'bn'} lang */
export function toLocalDigits(s, lang) {
  return lang === 'bn' ? s.replace(/[0-9]/g, (d) => BN[Number(d)]) : s;
}
```

```
/** Bangla digits to ASCII; other characters unchanged. @param {string} s */
export function toAsciiDigits(s) {
  return s.replace(/[<U+09E6>-<U+09EF>]/g, (c) => String(BN.indexOf(c)));
}

/** @param {string} digits */
function groupLakh(digits) {
  if (digits.length <= 3) return digits;
  let rest = digits.slice(0, -3);
  const parts = [digits.slice(-3)];
  while (rest.length > 2) { parts.unshift(rest.slice(-2)); rest = rest.slice(0, -2); }
  if (rest) parts.unshift(rest);
  return parts.join(',');
}

/** @param {bigint|number|string} v */
function toPoisha(v) {
  if (typeof v === 'bigint') return v;
  if (typeof v === 'number') {
    if (!Number.isSafeInteger(v)) throw new RangeError('poisha must be a safe integer');
    return BigInt(v);
  }
  if (typeof v === 'string' && /^-?\d+$/.test(v)) return BigInt(v);
  throw new TypeError('poisha must be an integer');
}

/**
 * Format an amount in poisha.
 * decimals 'auto' drops ".00" (single prices); 'always' keeps two decimals (breakdowns, receipts).
 * @param {bigint|number|string} poisha
 * @param {'en'|'bn'} [lang]
 * @param {{decimals?: 'auto'|'always', sign?: boolean}} [opts]
 */
export function formatMoney(poisha, lang = 'en', { decimals = 'auto', sign = false } = {}) {
  const p = toPoisha(poisha);
  const neg = p < 0n;
  const a = neg ? -p : p;
  const paisa = (a % 100n).toString().padStart(2, '0');
  let s = '<U+09F3>' + groupLakh((a / 100n).toString());
  if (decimals === 'always' || paisa !== '00') s += '.' + paisa;
  s = toLocalDigits(s, lang);
  return (neg ? '<U+2212>' : sign && p > 0n ? '+' : '') + s;
}

/**
 * Parse what a user typed as a taka amount (bid, price, partial refund) into poisha.
 * Accepts ASCII or Bangla digits, group commas, an optional taka sign, "Tk" or "BDT", and up to
 * two decimals. Never goes through a floating-point number.
 * @param {string} input
 * @returns {{ok: true, poisha: bigint} | {ok: false, reason: 'empty'|'format'|'decimals'|'too_large'}}
 */
export function parseTaka(input) {
  let s = toAsciiDigits(String(input)).trim().replace(/^(<U+09F3>|tk\.?|bdt)\s*/i, '').replace(/\s+/g, '');
  if (s === '') return { ok: false, reason: 'empty' };
  if (/^\d+\.\d{3,}$/.test(s.replace(/,/g, ''))) return { ok: false, reason: 'decimals' };
  // commas only as separators between digit groups (both 1,234,567 and 12,34,567 are accepted)
  if (!/^\d{1,3}(,\d{2,3})*(\.\d{1,2})?$|^\d+(\.\d{1,2})?$/.test(s)) return { ok: false, reason: 'format' };
  s = s.replace(/,/g, '');
  const [whole, frac = ''] = s.split('.');
  const poisha = BigInt(whole) * 100n + BigInt((frac + '00').slice(0, 2));
  if (poisha >= MAX_POISHA) return { ok: false, reason: 'too_large' };
  return { ok: true, poisha };
}
```

B.4 src/time.js

```
// TrustBazaar web client - Dhaka time, countdowns and the server clock (Unit 8 reference module).
// NFR-USE-08: every time is shown in Asia/Dhaka with its date; every deadline also as a countdown.
// FR-AUC-01: auction countdowns run on the server's clock, not the device's.

import { toLocalDigits } from './money.js';
```

```
export const TZ = 'Asia/Dhaka';

const fmt = {
  en: new Intl.DateTimeFormat('en-GB', { timeZone: TZ, weekday: 'short', day: 'numeric', month: 'short', year
     : 'numeric',
    hour: 'numeric', minute: '2-digit', hour12: true, numberingSystem: 'latn' }),
  bn: new Intl.DateTimeFormat('bn-BD', { timeZone: TZ, weekday: 'short', day: 'numeric', month: 'short', year
     : 'numeric',
    hour: 'numeric', minute: '2-digit', hour12: true, numberingSystem: 'beng' }),
};
const hourIn = new Intl.DateTimeFormat('en-GB', { timeZone: TZ, hour: 'numeric', hourCycle: 'h23' });

/**
 * "Tue␣20␣Oct␣2026,␣6:00␣pm" / "<U+09AE><U+0999><U+09CD><U+0997><U+09B2>␣<U+09E8><U+09E6>␣<U+0985><U+0995><U
     +09CD><U+099F><U+09CB>␣<U+09E8><U+09E6><U+09E8><U+09EC>,␣<U+09EC>:<U+09E6><U+09E6>␣<U+0985><U+09AA><U+09
     B0><U+09BE><U+09B9><U+09CD><U+09A3>", always in Dhaka time.
 * The day period is set here rather than taken from the browser, because ICU data differs
 * between browsers (some print "PM" inside Bangla text).
 * @param {number} ms epoch milliseconds @param {'en'|'bn'} lang
 */
export function formatDateTime(ms, lang = 'en') {
  const d = new Date(ms);
  const pm = Number(hourIn.format(d)) >= 12;
  const parts = fmt[lang].formatToParts(d);
  const get = (/** @type {string} */ t) => parts.find((p) => p.type === t)?.value ?? '';
  const period = lang === 'bn' ? (pm ? '<U+0985><U+09AA><U+09B0><U+09BE><U+09B9><U+09CD><U+09A3>' : '<U+09AA
     ><U+09C2><U+09B0><U+09CD><U+09AC><U+09BE><U+09B9><U+09CD><U+09A3>') : pm ? 'pm' : 'am';
  const s = `${get('weekday')}␣${get('day')}␣${get('month')}␣${get('year')},␣${get('hour')}:${get('minute')}␣
     ${period}`;
  return toLocalDigits(s, lang);
}

/**
 * Countdown text. Above a day: days and hours; above an hour: hours and minutes; below: m:ss.
 * @param {number} ms remaining milliseconds @param {'en'|'bn'} lang
 * @returns {{text: string, ended: boolean, urgent: boolean}}
 */
export function formatCountdown(ms, lang = 'en') {
  if (ms <= 0) return { text: lang === 'bn' ? '<U+09B6><U+09C7><U+09B7> <U+09B9><U+09AF><U+09BC><U+09C7><U
     +099B><U+09C7>' : 'Ended', ended: true, urgent: false };
  const s = Math.ceil(ms / 1000);
  const d = Math.floor(s / 86400), h = Math.floor((s % 86400) / 3600), m = Math.floor((s % 3600) / 60), sec =
      s % 60;
  const p2 = (/** @type {number} */ n) => String(n).padStart(2, '0');
  let text;
  if (d >= 1) text = lang === 'bn' ? `${d}␣<U+09A6><U+09BF><U+09A8>␣${h}␣<U+0998><U+09A3><U+09CD><U+099F><U
     +09BE>` : `${d}␣${d␣===␣1␣?␣'day'␣:␣'days'}␣${h}␣h`;
  else if (h >= 1) text = lang === 'bn' ? `${h}␣<U+0998><U+09A3><U+09CD><U+099F><U+09BE>␣${p2(m)}␣<U+09AE><U
     +09BF><U+09A8><U+09BF><U+099F>` : `${h}␣h␣${p2(m)}␣min`;
  else text = `${m}:${p2(sec)}`;
  return { text: toLocalDigits(text, lang), ended: false, urgent: s <= 300 };
}

/**
 * A deadline as both a time and a countdown (NFR-USE-08).
 * @param {number} deadlineMs @param {number} nowMs @param {'en'|'bn'} lang
 */
export function formatDeadline(deadlineMs, nowMs, lang = 'en') {
  const when = formatDateTime(deadlineMs, lang);
  const c = formatCountdown(deadlineMs - nowMs, lang);
  if (c.ended) return lang === 'bn' ? `${when}␣(${c.text})` : `${when}␣(${c.text})`;
  return lang === 'bn' ? `${when}␣(<U+0986><U+09B0>␣${c.text})` : `${when}␣(in␣${c.text})`;
}

/**
 * Estimates the server's clock from the serverTime field of responses and live messages.
 * Each HTTP sample bounds the offset on both sides: serverTime was read between sending and
 * receiving, so offset is in [serverTime - receivedAt, serverTime - sentAt]. A live message gives
 * only the lower bound, because it may have been delayed. The estimate is the middle of the
 * intersection of the recent samples; if they contradict each other (the device clock jumped),
 * only the newest sample is kept.
 */
export class ServerClock {
  /** @param {{window?: number, now?: () => number}} [o] */
```

```
  constructor({ window = 8, now = () => Date.now() } = {}) {
    this.window = window;
    this.local = now;
    /** @type {{lo: number, hi: number}[]} */
    this.samples = [];
    this.estimate = 0;
  }
  /** @param {number} serverTime @param {number} sentAt @param {number} receivedAt */
  sampleHttp(serverTime, sentAt, receivedAt) {
    this.#add({ lo: serverTime - receivedAt, hi: serverTime - sentAt });
  }
  /** @param {number} serverTime @param {number} receivedAt */
  sampleMessage(serverTime, receivedAt) {
    this.#add({ lo: serverTime - receivedAt, hi: Infinity });
  }
  /** @param {{lo: number, hi: number}} s */
  #add(s) {
    this.samples.push(s);
    if (this.samples.length > this.window) this.samples.shift();
    let lo = -Infinity, hi = Infinity;
    for (const x of this.samples) { lo = Math.max(lo, x.lo); hi = Math.min(hi, x.hi); }
    if (lo > hi) { this.samples = [s]; lo = s.lo; hi = s.hi; }
    this.estimate = Number.isFinite(hi) ? (lo + hi) / 2 : lo;
  }
  /** offset in ms to add to the device clock */
  get offset() { return this.estimate; }
  /** the server's current time, estimated */
  now() { return this.local() + this.estimate; }
}
```

B.5 src/moneyState.js

```
// TrustBazaar web client - the order's money in plain language (Unit 8 reference module).
// NFR-USE-07: the order view says where the money is, who gets it, when, and what can stop it.
// The function picks a message and its values; the screen renders it with the catalogue.

import { formatMoney } from './money.js';
import { formatDateTime } from './time.js';

/**
 * @typedef {object} OrderMoneyView  fields of GET /orders/{id} (Unit 2)
 * @property {'awaiting_payment'|'cancelled'|'paid'|'closed'} status
 * @property {'held'|'frozen_disputed'|'released'|'refunded'|'partially_refunded'|null} escrowState
 * @property {'standard'|'game_protection'} releasePolicy
 * @property {number} total                 poisha the buyer pays or paid
 * @property {number} [refunded]            poisha refunded so far
 * @property {number} [paymentDueAt]
 * @property {number} [fulfilmentDueAt]
 * @property {number|null} [deliveryEventAt]
 * @property {number|null} [autoReleaseAt]  for game accounts: the end of the protection period
 * @property {number|null} [closedAt]
 */

/**
 * @param {OrderMoneyView} o @param {'buyer'|'seller'} role
 * @returns {{key: string, vars: Record<string, {money: number} | {time: number}>}}
 */
export function describeMoney(o, role) {
  const k = (/** @type {string} */ s) => `money.${s}.${role}`;
  if (o.status === 'awaiting_payment') return { key: k('awaiting_payment'), vars: { due: { time: o.
     paymentDueAt ?? 0 } } };
  if (o.status === 'cancelled') return { key: k('cancelled_unpaid'), vars: {} };
  switch (o.escrowState) {
    case 'held':
      if (!o.deliveryEventAt) {
        return { key: k('held_before_delivery'), vars: { amount: { money: o.total }, due: { time: o.
     fulfilmentDueAt ?? 0 } } };
      }
      if (o.releasePolicy === 'game_protection') {
        return { key: k('held_protection'), vars: { at: { time: o.autoReleaseAt ?? 0 } } };
      }
      return { key: k('held_after_delivery'), vars: { at: { time: o.autoReleaseAt ?? 0 } } };
```

```
    case 'frozen_disputed':
      return { key: k('frozen'), vars: { amount: { money: o.total } } };
    case 'released':
      return { key: k('released'), vars: { amount: { money: o.total }, at: { time: o.closedAt ?? 0 } } };
    case 'refunded':
      return { key: k('refunded'), vars: { amount: { money: o.refunded ?? o.total }, at: { time: o.closedAt
     ?? 0 } } };
    case 'partially_refunded':
      return { key: k('partially_refunded'), vars: { amount: { money: o.refunded ?? 0 }, at: { time: o.
     closedAt ?? 0 } } };
    default:
      throw new Error(`unknown␣escrow␣state␣${o.escrowState}`);
  }
}

/**
 * @param {(key: string, vars?: Record<string, string>) => string} t
 * @param {{key: string, vars: Record<string, {money: number} | {time: number}>}} d
 * @param {'en'|'bn'} lang
 */
export function renderMoney(t, d, lang) {
  /** @type {Record<string, string>} */
  const vars = {};
  for (const [name, v] of Object.entries(d.vars)) {
    vars[name] = 'money' in v ? formatMoney(v.money, lang) : formatDateTime(v.time, lang);
  }
  return t(d.key, vars);
}

/** every key describeMoney can return, for the catalogue test */
export const MONEY_KEYS = ['awaiting_payment', 'cancelled_unpaid', 'held_before_delivery', 'held_protection',
  'held_after_delivery', 'frozen', 'released', 'refunded', 'partially_refunded']
  .flatMap((s) => [`money.${s}.buyer`, `money.${s}.seller`]);
```

B.6 src/live.js

```
// TrustBazaar web client - ordering of live updates (Unit 8 reference module).
// Auction messages carry the whole state and a sequence number (Unit 3): keep the newest, drop the
// rest, and fetch /state after a reconnection. Chat messages carry a per-thread sequence number
// (Unit 6): append in order, recognise our own resends by client id, and resync after a gap.

/**
 * @typedef {{seq: number, price: number, endAt: number, serverTime: number}} AuctionState
 */
export class AuctionFeed {
  constructor() { /** @type {AuctionState | null} */ this.state = null; }
  /** @param {AuctionState} s @returns {boolean} whether the view must update */
  apply(s) {
    if (this.state && s.seq <= this.state.seq) return false;
    this.state = s;
    return true;
  }
}

/**
 * @typedef {{seq: number, clientMsgId: string, senderId: string, text?: string, flags?: string[]}}
     ChatMessage
 */
export class ChatThread {
  constructor() {
    /** @type {ChatMessage[]} */ this.messages = [];
    /** @type {Map<string, {clientMsgId: string, text: string}>} sent, not yet confirmed */ this.pending =
     new Map();
    this.lastSeq = 0;
    this.needsResync = false;
  }
  /** @param {string} text @param {string} clientMsgId a random id kept for the resends of one message */
  send(text, clientMsgId) { this.pending.set(clientMsgId, { clientMsgId, text }); }

  /** a message pushed over the socket @param {ChatMessage} m */
  receive(m) {
    if (m.seq <= this.lastSeq) return;                  // already have it
```

```
    if (m.seq > this.lastSeq + 1) { this.needsResync = true; return; }   // gap: fetch ?after=lastSeq
    this.#append(m);
  }

  /** the result of GET /threads/{id}/messages?after=lastSeq @param {ChatMessage[]} list */
  resync(list) {
    for (const m of [...list].sort((a, b) => a.seq - b.seq)) {
      if (m.seq === this.lastSeq + 1) this.#append(m);
    }
    this.needsResync = false;
  }

  /** @param {ChatMessage} m */
  #append(m) {
    this.messages.push(m);
    this.lastSeq = m.seq;
    this.pending.delete(m.clientMsgId);
  }
}
```

B.7 src/i18n.js and src/redirect.js

```
// TrustBazaar web client - message catalogues (Unit 8 reference module).
// Messages are plain text with {name} placeholders. No catalogue entry may contain markup:
// everything is rendered as text (SEC-INP-03).

/** @typedef {'en'|'bn'} Lang */

/**
 * @param {Record<string, string>} catalogue
 * @param {{strict?: boolean}} [o] strict: throw on a missing key or variable (tests and development)
 */
export function createT(catalogue, { strict = false } = {}) {
  /** @param {string} key @param {Record<string, string>} [vars] */
  return function t(key, vars = {}) {
    const msg = catalogue[key];
    if (msg === undefined) {
      if (strict) throw new Error(`missing␣message␣${key}`);
      return key;
    }
    return msg.replace(/\{(\w+)\}/g, (_, name) => {
      if (!(name in vars)) {
        if (strict) throw new Error(`message␣${key}␣needs␣{${name}}`);
        return '';
      }
      return vars[name];
    });
  };
}

/** @param {string} msg */
export function placeholders(msg) {
  return [...msg.matchAll(/\{(\w+)\}/g)].map((m) => m[1]).sort();
}
```

```
// TrustBazaar web client - leaving the site for a payment provider (Unit 8 reference module).
// The backend returns the provider's page to open (Unit 2). The client follows it only to an exact,
// configured provider host over HTTPS, or to a same-origin path (the mock provider in the academic
// deliverable). A changed or injected response therefore cannot send the buyer to a look-alike page.
// The host list comes from the build configuration, filled from Unit 2's adapter settings.

/**
 * @param {string} target the URL from the API
 * @param {string[]} allowedHosts exact host names, e.g. the providers' sandbox hosts
 * @param {string} origin location.origin
 * @returns {URL | null} the URL to navigate to, or null if it must not be followed
 */
export function checkProviderRedirect(target, allowedHosts, origin) {
  let u;
  try { u = new URL(target, origin); } catch { return null; }
  if (u.origin === origin) return u.pathname.startsWith('/') && !u.pathname.startsWith('//') ? u : null;
```

```
  if (u.protocol !== 'https:') return null;
  if (u.username || u.password) return null;
  if (u.port && u.port !== '443') return null;
  return allowedHosts.includes(u.hostname.toLowerCase()) ? u : null;
}
```

B.8 src/messages/en.json

```
{
  "money.awaiting_payment.buyer": "Nothing has been charged yet. Pay by {due} to keep this order.",
  "money.awaiting_payment.seller": "Waiting for the buyer to pay by {due}. Do not hand over the item yet.",
  "money.cancelled_unpaid.buyer": "This order was cancelled before payment. Nothing was charged.",
  "money.cancelled_unpaid.seller": "This order was cancelled before payment. There is nothing to hand over.",
  "money.held_before_delivery.buyer": "Your payment of {amount} is held by TrustBazaar. The seller must hand
     the item over by {due}, or you get a full refund.",
  "money.held_before_delivery.seller": "The buyer's payment is held by TrustBazaar. Hand the item over by {
     due} to be paid. If you miss it, the buyer is refunded.",
  "money.held_after_delivery.buyer": "Your payment is held by TrustBazaar. The seller is paid when you
     confirm receipt, or automatically on {at} unless you open a dispute.",
  "money.held_after_delivery.seller": "The item was delivered. You are paid when the buyer confirms receipt,
     or automatically on {at} if no dispute is opened.",
  "money.held_protection.buyer": "Your payment is held until the protection period ends on {at}. If the
     seller takes the account back before then, open a dispute.",
  "money.held_protection.seller": "You are paid on {at}, when the protection period ends, unless the buyer
     opens a dispute.",
  "money.frozen.buyer": "Your payment of {amount} is frozen while TrustBazaar reviews your dispute. Nobody is
      paid until it is decided.",
  "money.frozen.seller": "The payment of {amount} is frozen while TrustBazaar reviews the buyer's dispute.
     You are paid only if it is decided in your favour.",
  "money.released.buyer": "Your payment of {amount} was released to the seller on {at}.",
  "money.released.seller": "The payment of {amount} was released to you on {at}. See Payouts for the transfer
     .",
  "money.refunded.buyer": "You were refunded {amount} on {at}.",
  "money.refunded.seller": "The buyer was refunded {amount} on {at}. You receive nothing for this order.",
  "money.partially_refunded.buyer": "You were refunded {amount} on {at}. The rest was paid to the seller.",
  "money.partially_refunded.seller": "The buyer was refunded {amount} on {at}. You were paid the rest.",

  "confirm.confirm_receipt": "Releasing payment pays the seller {amount} now and ends your right to dispute
     this order.",
  "confirm.release_early": "Releasing early pays the seller {amount} now. You can no longer dispute if the
     seller takes the account back, even before {at}.",
  "confirm.open_dispute": "Opening a dispute freezes the payment until TrustBazaar decides. You can withdraw
     it before the decision, but not open the same kind of dispute again.",
  "confirm.withdraw_dispute": "Withdrawing ends this dispute for good. The seller is then paid automatically
     on {at} unless you open a different kind of dispute.",
  "confirm.seller_cancel": "Cancelling refunds the buyer {amount} in full and gives you a seller strike.
     Three strikes in 90 days suspend your selling.",
  "confirm.place_bid": "A bid of {amount} is binding. If you win, you must pay within the payment window or
     receive a strike.",
  "confirm.proxy_bid": "TrustBazaar will bid for you up to {amount}, only as much as needed to keep you in
     the lead. Every bid it places is binding.",
  "confirm.buy_now": "You will pay {amount} in total: item {item} and delivery {delivery}. The money is held
     until the item is delivered.",
  "confirm.share_handover_code": "Give this code to the seller only after you have checked the item. Entering
      it counts as delivery.",

  "sim.payment": "Test mode: no real money is moved. Payments are simulated.",
  "sim.courier": "Test mode: this shipment is simulated. No courier will collect it.",

  "error.network": "We could not reach TrustBazaar. Check your connection and try again.",
  "error.rate_limited": "Too many attempts. Try again in {seconds} seconds.",
  "error.session_ended": "You have been signed out. Sign in again to continue.",
  "error.step_up": "For your security, enter the code we sent to your phone to continue.",
  "error.forbidden": "You are not allowed to do this.",
  "error.not_found": "This page or item no longer exists.",
  "error.conflict": "This changed while you were looking at it. The page has been refreshed.",
  "error.generic": "Something went wrong on our side. If you contact us, quote reference {ref}."
}
```

B.9 `container/nginx.conf` and the credential pages' headers

```
# TrustBazaar web container (Unit 8 reference): serves the four documents with their own headers,
# and forwards the API and the live connection to the backend, so browser and API share one origin.
# BACKEND is replaced at start (docker: backend:3000). TLS and HSTS are added by the TLS front (Unit 9).

map $http_upgrade $connection_upgrade { default upgrade; '' close; }

server {
  listen       8080;
  server_name  _;
  root         /usr/share/nginx/html;
  server_tokens off;
  charset utf-8;
  client_max_body_size 12m;          # CP-33 images (10 MB) plus form overhead

  # ------------------------------------------------------------------ API and live connection
  location /api/ {
    proxy_pass         http://BACKEND;
    proxy_http_version 1.1;
    proxy_set_header   Host              $host;
    proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
    proxy_set_header   X-Forwarded-Proto $scheme;
    proxy_set_header   Origin            $http_origin;        # Unit 1's CsrfGuard checks it
    proxy_read_timeout 60s;
  }
  location /socket.io/ {
    proxy_pass         http://BACKEND;
    proxy_http_version 1.1;
    proxy_set_header   Upgrade    $http_upgrade;
    proxy_set_header   Connection $connection_upgrade;
    proxy_set_header   Host       $host;
    proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_read_timeout 120s;                                  # longer than Socket.IO's ping interval
  }

  # ------------------------------------------------------------------ health check for Compose (Unit 9)
  location = /healthz { access_log off; default_type text/plain; return 200 "ok\n"; }

  # ------------------------------------------------------------------ hashed build output
  location /assets/ {
    try_files $uri =404;
    add_header Cache-Control "public, max-age=31536000, immutable" always;
    add_header X-Content-Type-Options nosniff always;
    add_header Cross-Origin-Resource-Policy same-origin always;
  }

  # ------------------------------------------------------------------ the four documents
  # (add_header in a location replaces the parent's, so each document lists all of its headers)

  # staff console (FR-OPS, SEC-SES-06): no camera, no location, never cached
  location /staff/ { try_files /staff.html =404; include /etc/nginx/tb/staff-headers.conf; }
  location /admin/ { try_files /staff.html =404; include /etc/nginx/tb/staff-headers.conf; }
  location = /staff.html { include /etc/nginx/tb/staff-headers.conf; }

  # KYC capture: the only document allowed the camera (SEC-WEB-02, FR-KYC-02)
  location = /account/kyc  { try_files /kyc.html =404; include /etc/nginx/tb/kyc-headers.conf; }
  location /account/kyc/   { try_files /kyc.html =404; include /etc/nginx/tb/kyc-headers.conf; }
  location = /kyc.html { include /etc/nginx/tb/kyc-headers.conf; }

  # credential pages (SEC-GAM-03): the strictest policy, never stored
  location /secure/ { try_files /secure.html =404; include /etc/nginx/tb/secure-headers.conf; }
  location = /secure.html { include /etc/nginx/tb/secure-headers.conf; }

  # the user app: every other path is a route of index.html
  location = /index.html { include /etc/nginx/tb/app-headers.conf; }
  location / {
    try_files $uri /index.html;
    include /etc/nginx/tb/app-headers.conf;
  }
}
```

```
# secure document headers (Unit 8, Section "Security headers per document")
```

```
add_header Content-Security-Policy "default-src 'none'; script-src 'self'; style-src 'self'; img-src 'self';
     font-src 'self'; connect-src 'self'; base-uri 'none'; form-action 'none'; frame-ancestors 'none'; require
     -trusted-types-for 'script'; trusted-types 'none'" always;
add_header Permissions-Policy "camera=(), geolocation=(), accelerometer=(), autoplay=(), bluetooth=(),
     display-capture=(), gyroscope=(), hid=(), magnetometer=(), microphone=(), payment=(), serial=(), usb=()"
     always;
add_header Cache-Control "no-store" always;
add_header Referrer-Policy no-referrer always;
add_header X-Content-Type-Options nosniff always;
add_header Cross-Origin-Opener-Policy same-origin always;
add_header Cross-Origin-Resource-Policy same-origin always;
```